c9r (CanonFodder) — Requirements & Design Documentation

Version 0.8 — 2026-02-28

B1 Requirements Specification

B1.1 Purpose

c9r (CanonFodder) is a reproducible pipeline that ingests music listening events (scrobbles), resolves artist-name ambiguity through record linkage and disambiguation, enriches data with metadata and delivers interactive analytics. A trained binary classifier supports user-reviewed canonisation recommendation for artist name variants.

B1.2 Scope

In scope: automated data ingestion from ListenBrainz and Last.fm; MusicBrainz enrichment (local Docker mirror or remote API); manual and ML-assisted canonisation via a Click CLI; Parquet-native storage with DuckDB as the query engine; versioned Parquet schemas; Prefect workflow orchestration; model training and serving for artist-name record linkage; prediction logging with feature vectors for drift detection; Docker packaging.

Out of scope: streaming playback, real-time recommendation engine, paid cloud hosting, Wikipedia-style disambiguation of distinct same-name artists (planned for v1).

B1.3 Stakeholders

- Primary user: analyse personal listening history with clean, canonised data.
- Occasional listener: insight into listening patterns via dashboards and profiling.
- Integration partners (LB, platforms): optional data exchange.

B1.4 Overall Description

c9r is a pull-based pipeline triggered manually or via Prefect. It converts raw scrobbles into clean, year-partitioned Parquet files, enriches with metadata, trains and serves a record-linkage classifier, and logs predictions for drift monitoring. Users interact through a Click CLI that exposes data gathering, canonisation review, model inference, profiling, QA, and schema management capabilities.

B1.5 Assumptions, Dependencies, Constraints

- Python >= 3.12, PEP 8, 80% test coverage target.
- External APIs: ListenBrainz (LB_TOKEN) and MusicBrainz (rate-limited at 1 req/s or local PostgreSQL mirror via musicbrainz-docker).
- Local filesystem read/write for Parquet cache.
- Docker container run via: docker run c9r.

B2 Software Requirements Document

B2.1 System Interfaces

- MusicBrainz API / local mirror: artist lookup, search, alias resolution.
- DuckDB: analytical query engine over Parquet files.

- FastAPI + Uvicorn: serve LightGBM classifier via ASGI endpoint.
- MLflow: experiment tracking with SQLite backend.
- Optuna: hyperparameter tuning with precision-biased objective.

B2.2 Functional Requirements

- FR-01 Fetch scrobbles — Pull recent tracks since last stored timestamp; persist to year-partitioned scrobble Parquet store.
- FR-02 Normalise scrobbles — Rename columns, convert UTS to UTC datetime, validate MBIDs, deduplicate.
- FR-03 Bulk insert — Append normalised rows to Parquet file with application-layer dedup.
- FR-04 Artist enrichment — For new MBIDs, fetch country & aliases from MB; cache in artist_info.parquet.
- FR-05 Canonisation — Group artist name variants, store mapping in AVC, apply to scrobble history.
- FR-06 Gold standard — Build and curate labelled variant pairs (gs_mb.parquet) from local MB mirror.
- FR-07 Model training — Train binary classifier on gold standard with k-fold CV and MLflow tracking.
- FR-08 Model serving — Expose model via ASGI endpoint for canonisation recommendations.
- FR-09 BI middleend — Dashboards and profiling via Click CLI subcommands.
- FR-10 Retry & backoff — Network requests retry up to 8 times with exponential backoff (2^n, capped at 120s).
- FR-11 Workflow orchestration — One-click Prefect flow (cf_ingest) covering FR-01 through FR-07 with 8 tasks.
- FR-12 Docker distro — Build image with code, dependencies, model artefacts, and trained pickle.
- FR-13 Drift detection — Log predictions (timestamp, pair, probability, feature vector) to predictions_log.parquet; QA compares baseline vs recent windows on mean probability shift (threshold: 0.10), ambiguous-band proportion (threshold: 2x), and per-feature median quantile shift (threshold: 0.15).
- FR-14 Schema management — Versioned Parquet schemas with auto-stamp on write, validate on read, and CLI migration commands.

B2.3 Performance Requirements

- Ingest 10,000 scrobbles per minute on consumer-grade hardware.
- End-to-end Prefect flow completes within 15 minutes for 1 million scrobbles.
- Dashboard renders top-10 artists within 2 seconds via DuckDB over Parquet.

B2.4 Data Requirements

- All timestamps stored in timezone-aware UTC.
- Parquet files partitioned by year for the scrobble table.
- Primary key semantics enforced at the application layer (unique composite of artist_name + track_title + play_time).
- Parquet compression: zstd.
- Versioned schemas stamped in Parquet file metadata.

B2.5 Reliability & Availability

- Retry logic on HTTP 5xx and network errors (FR-10).
- Schema changes tracked via versioned Parquet schemas with migration scripts.
- Unit and integration tests; CI fails if coverage < 80% or linting errors.

B2.6 Security & Privacy

- Secrets stored in .env; mounted via Docker secrets in production.
- Read-only LB scope.

- GDPR compliance: user can purge all personal data via CLI command (c9r purge --all --yes).

B2.7 Acceptance Criteria

- All functional requirements demonstrably fulfilled.
- Ingest 300k scrobbles without duplication.
- Dashboard renders top-10 artists within 2 seconds.
- Classifier exceeds threshold on custom c9r score (0.4 x HiP_P + 0.3 x HiP_F1 + 0.3 x AUC) with emphasis on precision.
- Canonised top-artist ranking matches a manually verified ground truth.

B3 High-Level Design

B3.1 Architectural Overview

The architecture follows a medallion-style pattern with layers:

1. Source layer — fetches raw data from ListenBrainz, Last.fm, and MusicBrainz APIs (or local Docker mirror) via resilient HTTP clients.
2. ELT layer — normalises, deduplicates, fixes encoding, enriches, and canonises data, writing results as year-partitioned Parquet files.
3. Analytical layer — DuckDB queries Parquet files directly for dashboards, profiling, and QA reporting. Canonical name resolution via artist_info aliases CTE.
4. ML layer — feature engineering (3-tier + interaction + catalogue), training with k-fold CV and MLflow, Optuna tuning, and a serving sidecar (FastAPI).
5. Monitoring layer — prediction logging with full feature vectors; drift detection comparing baseline vs recent windows on probability and feature quantile distributions.

B3.2 Architectural Decisions

- Parquet + DuckDB: columnar, zstd-compressed, type-preserving, predicate pushdown. DuckDB runs in-process. Transactional guarantees handled at the application layer via Pandas.
- Click: composable command groups, autogenerated help, parameter validation, cross-platform.
- Prefect 3.x: batch-oriented, single-user workflow model with exponential backoff retries.
- Local MusicBrainz mirror: unlimited query throughput for bulk enrichment, bypassing the 1 req/s API limit.
- LightGBM: fast inference, small footprint, native GPU support, well-maintained. Self-contained sklearn Pipeline (RobustScaler -> LGBMClassifier) carries its own column order in feature_names_in_.
- Prediction logging: every canon machine run persists timestamp, variant pair, probability, and the full ~63-feature JSON vector to predictions_log.parquet for downstream drift detection.

B3.3 Key Components

- HTTP clients: resilient GET with tenacity retries, dual-source (LB + Last.fm), MB API with 1 req/s rate limiting.
- ELT pipeline: normalisation, canonisation, enrichment, gold-standard generation.
- Parquet store: scrobble/ (year-partitioned), artist_info, avc, c, gs_mb, predictions_log, qa_report, uc.
- DuckDB in-process: SQL over Parquet for dashboards, profiling, and analytical queries.
- ML pipeline: feature engineering, training, evaluation, model persistence. Optuna tuning with precision-biased objective.
- ASGI endpoint: FastAPI serving predictions for canonisation recommendations.

- Prefect flow: 8 tasks — fetch, fix-encoding, enrich, clean, canonise, propagate, augment, retrain.
- Click CLI: dashboard, profile, canon, qa, train, tune, schema, and maintenance subcommands.

B3.4 Data Flow

1. Prefect triggers the fetch-and-ingest task.
2. LBClient/lfAPI pulls new scrobbles, returns JSON.
3. ELT layer converts to DataFrame, fixes encoding, deduplicates, appends to PQ/scrobble/year=YYYY/ partitions.
4. MB enrichment fetches missing artist rows; upserts into artist_info.parquet.
5. Canonisation (ML batch or interactive CLI) groups variants, updates avc.parquet.
6. Every ML prediction is logged to predictions_log.parquet with the full feature vector.
7. Propagation applies decided AVC mappings to artist_info.
8. User opens dashboard or profile; DuckDB queries Parquet files on the fly.
9. QA checks validate data quality and monitor prediction drift.

B3.5 Data Schema

- scrobble (fact, year-partitioned): artist_name, album_title, track_title, artist_mbid, play_time. Unique: (artist_name, track_title, play_time).
- artist_info (dimension): artist_name, mbid, country, disambiguation_comment, aliases.
- avc (dimension): artist_variants_hash, canonical_name, to_link, comment, stamp, artist_variants_text.
- c (dimension): ISO-2, ISO-3, en_name, hu_name.
- uc (dimension): country_code, start_date, end_date. Period check on dates.
- gs_mb (training): variant_a, variant_b, to_link, source.
- predictions_log (monitoring): timestamp, variant_a, variant_b, probability, features_json.
- qa_report (operational): timestamp, source, target, passed, and check-specific fields.

B3.6 Technology Stack

- Python 3.12+: modern typing, datetime.UTC, f-strings.
- Apache Parquet: columnar, portable, efficient, zstd-compressed.
- DuckDB: in-process OLAP; native PQ; SQL over Parquet.
- Prefect 3.x: Py-native, local dev, no server required for local runs.
- Click: composable commands, cross-platform.
- LightGBM + scikit-learn: tree-based classification, pipeline-based.
- FastAPI + Uvicorn: async, lightweight, auto-generated OpenAPI docs.
- MLflow: experiment tracking, metric/artefact logging.
- Optuna: hyperparameter optimisation with precision-biased objective.
- Local MB mirror: bypass MB API rate limit for bulk enrichment.

B4 Low-Level Design

B4.1 Module Breakdown

- corefunc/canon/trainer.py: build_training_data(), compute_all_features(), run_training() — unified pipeline: AVC -> pairs -> WRatio filter -> k-fold CV -> MLflow.

- corefunc/canon/workflow.py: discover_candidates(), write_new_candidates(), propagate_avc(), avc_summary() — AVC CRUD + ML-driven candidate discovery. Logs predictions with feature vectors.

- corefunc/canon/tuner.py: Optuna precision-biased hyperparameter search for LightGBM/XGBoost/ExtraTrees.

- corefunc/canon/experiment_runner.py: multi-model benchmarking with 8 models, nested MLflow runs.

- corefunc/canon/tcn_trainer.py: Siamese and Hybrid TCN architectures.

- corefunc/canon/augment.py: gold-standard pair extraction from local MB mirror.

- corefunc/enrich.py: enrich_artist_country(), backfill_mbids(), enrich_all() — orchestrates local MB / remote MB / Last.fm enrichment backends.

- corefunc/model_server.py: predict(), predict_batch(), health() — FastAPI ASGI serving.

- corefunc/profile.py: overview_stats(), variant_candidates(), top_artists_profile(), country_breakdown(), monthly_summary(), streak_analysis(), listening_clock_profile(), user_country_medal_profile() — DuckDB + RapidFuzz text profiling.

- corefunc/qa.py: qa_lb_ingest(), qa_artist_info(), qa_avc(), qa_gs_mb(), qa_uc(), qa_predictions() — schema/null/timestamp/dup/encoding checks + prediction drift detection with feature quantile comparison. Persists reports to qa_report.parquet.

- flows/cf_ingest.py: Prefect flow with 8 tasks — ingest -> fix-encoding -> enrich -> clean -> canonise -> propagate -> augment -> retrain. Exponential backoff retries.

- helpers/cli.py: Click prompts for manual grouping, user-country timeline editing.

- helpers/features.py: compute_pair_features() — three-tier pairwise features (whole-string / token / character).

- helpers/inference.py: compute_inference_features(), load_model() — catalogue-enriched features (disco + melo) for serving; session-level catalogue cache.

- helpers/io.py: read_parquet(), dump_parquet(), append_to_parquet(), ingest_scrobbles(), normalise_scrobble_df(), read_scrobble_df(), dump_scrobble_df(), scrobble_duckdb_from() — dedup-aware Parquet I/O with schema-stamped writes via PyArrow.

- helpers/query.py: top_artists(), top_albums(), top_tracks(), scrobble_count(), monthly_scrobble_counts(), yearly_top_n_artists(), listening_clock(), daily_scrobble_dates(), artist_country_stats() — DuckDB analytical queries with canonical-name resolution via aliases CTE.

- helpers/schema.py: stamp_metadata(), validate_schema(), migrate_file() — versioned Parquet metadata with decorator-based migration framework.

- helpers/stats.py: length_stats(), cramers_v(), variance_testing() — feature engineering + evaluation support.

- HTTP/client.py: resilient HTTP GET with exponential backoff (2^n, capped at 120s).

- HTTP/lblink.py: LBClient, export_listens_to_parquet(), fetch_scrobbles_since() — dual backend (pylistenbrainz / plain requests); token from .env.

- HTTP/lfAPI.py: lastfm_request(), fetch_scrobbles_since(), enrich_artist_mbids(), sync_user_country() — paginated Last.fm API with signed requests.

- HTTP/mbAPI.py: lookup_mb_for(), fetch_country(), search_artist(), get_complete_artist_info() — tenacity retry; 1 req/s rate limit; artist cache dict.

B4.2 Core Data Model

Primary fact record:

```
SCROBBLE_SCHEMA = pa.schema([
    ("artist_name", pa.string()),
    ("album_title", pa.string()),
    ("track_title", pa.string()),
    ("artist_mbid", pa.string()),
    ("play_time", pa.timestamp("us", tz="UTC")),
])
```

B4.3 Algorithms

Canonisation:

- Classifier zoo: 5 base tree models (XGBoost, LightGBM, ExtraTrees, RandomForest, GradientBoosting) + 3 composites (Voting, Stacking, Bagging).
- TCNs with two architectures: Siamese and Hybrid (via tcn_trainer.py).
- Optuna tuning with precision-biased objective (c9r tune).
- MLflow tracking with k-fold stratified CV, multiple operating points (default 0.5, F1-optimal, high-precision P>=0.80), and nested runs.
- DBSCAN + manual anchors path as one of several data-source options (avc, mbdb, dbscan, dbscan-capped, mixed).
- Custom c9r score for model selection: 0.4 x HiP_P + 0.3 x HiP_F1 + 0.3 x AUC.

Feature engineering:

- Tier A (10 whole-string): 6 RapidFuzz scores + normalised Levenshtein + Jaro-Winkler + length_ratio + abs_len_diff.
- Tier B (5 token-level): token_count_diff, token_jaccard, shared_token_ratio, LCS token length, Kendall tau token-order displacement.
- Tier C (8 character-level): bigram/trigram Jaccard, edit operation breakdown (insert/delete/replace), shared prefix/suffix length, Unicode script mismatch flag.
- Interaction (30): pairwise diffs and products among the 6 similarity scores.
- Catalogue (10): 5 presence-and-quality features per domain (discography + melography) — using unified MBDB + scrobble fallback lookups.
- Pruning: variance (0.001) + Spearman correlation (0.95) iterative drop. ~63 raw features pruned to 41.

Bulk insert:

ingest_scrobbles() uses year-partitioned Parquet — each year partition is independently deduped and written.

Drift detection:

qa_predictions() compares baseline (default: 30 days) vs recent (default: 7 days) windows. _feature_quantile_drift() parses features_json, computes per-feature medians, and flags features whose absolute median shift exceeds 0.15.

B4.4 Error Handling & Logging

HTTP client retries up to 8 with exponential backoff (2^n, capped at 120s); logs to stdout. Non-fatal errors escalate via raise_for_status in ingest context. Parquet schema mismatches abort with clear error message. Stale schema versions emit warnings; future versions are rejected.

B4.5 Configuration

.env parsed by python-dotenv; required keys validated at startup. Default fallback uses local Parquet files with no external dependencies for tests. Environment variables: LASTFM_API_KEY, LB_TOKEN, LASTFM_USER, LB_USER, C9R_SOURCE.

B4.6 Testing Strategy

pytest suite in tests/ with fixtures (conftest.py creates temp PQ/ dirs and monkeypatches all path constants). CI matrix: unit, integration with temporary Parquet files, coverage gate >= 80%.

B5 Traceability Matrix

- B1.2 Scope -> ingest, enrich, canonise, profile, QA, schema management, drift detection.
- B1.3 Problem statement -> record linkage via ML-assisted binary classification.
- B1.7 Constraints -> network retry (FR-10), exponential backoff.
- B1.6 Description -> orchestration via Prefect (FR-11), 8 tasks.
- FR-12 -> Docker packaging with model pickle.
- FR-13 -> Drift detection via predictions_log.parquet and qa_predictions().
- FR-14 -> Schema management via helpers/schema.py.

B6 Open Issues, Future Work

- OAuth flow for ListenBrainz.
- Real-time streaming ingest (re-evaluate Kafka at that point).
- Expanded gold standard for multilingual artist names.
- Autolink tier (P >= 0.98) once training data grows sufficiently.
- Re-tuning with Optuna after substantial data expansion or programmatic bridge of MBDB-scrobbleDB gap.